

Declaración de Uso de Inteligencia Artificial Generativa

Proyecto: Goya Ledger (incluye LexChain y pruebas formales Lean 4) **Fecha:** 2026-09-04 **Autor responsable:** Clemente Falcone (clementeaf) **Versión del documento:** 2.0

1. Herramienta utilizada

Campo	Valor
Herramienta	Claude Code (CLI)
Proveedor	Anthropic, PBC
Modelos utilizados	Claude Opus 4, Claude Sonnet 4
Período de uso	Agosto 2026 – presente
Otras herramientas IA	Ninguna. No se utilizó GitHub Copilot, ChatGPT, Gemini ni otros asistentes.

2. Alcance del uso

El 100% del código fuente fue desarrollado mediante pair-programming asistido por Claude Code. Esto abarca:

- **Goya Ledger** (núcleo blockchain): ~125.500 líneas de Rust
- **LexChain** (motor de contratos legales): módulo integrado en `src/`
- **Pruebas formales** (Lean 4): ~700 líneas en `formal/`
- **Documentación técnica y de compliance:** `docs/`

La herramienta fue utilizada como asistente de desarrollo bajo dirección humana continua, no como generador autónomo.

3. Rol del autor humano

El autor humano único ejerció control sobre:

- Arquitectura del sistema y decisiones de diseño
- Definición de requisitos y especificaciones

- Selección de algoritmos y protocolos (PQC, BFT, DPoS)
- Revisión y aprobación de cada cambio antes de commit
- Definición de la estructura de módulos y API
- Criterios de compliance (FIPS 140-3, eIDAS, EA-103, Ley 19.799)
- Estrategia de testing y verificación formal

4. Trazabilidad

Evidencia	Detalle
Total de commits	791
Autores en git	1 (clementeaf)
Commits con trailer Co-Authored-By: Claude	5 (commits recientes)
Repositorio	Privado, acceso restringido
Historial de sesiones	Registrado en trailers Claude-Session

Los commits anteriores a la adopción del trailer formal fueron igualmente asistidos por Claude Code pero no llevan la anotación por convención adoptada posteriormente.

5. Posición legal

5.1 Propiedad del output

Los Términos de Servicio de Anthropic (vigentes a septiembre 2026) establecen que todo output generado por Claude es propiedad del usuario. No existe cláusula de cesión, licencia, regalía ni retención de derechos por parte del proveedor sobre el código producido.

5.2 Riesgo de IP de terceros

Riesgo evaluado: inexistente. A diferencia de herramientas de autocompletado basadas en repositorios indexados (ej. GitHub Copilot), Claude Code genera respuestas a partir del prompt y contexto local del usuario, no mediante recuperación verbatim de código de terceros. No existe riesgo de incorporación involuntaria de código bajo licencias restrictivas (GPL, AGPL, etc.).

5.3 Registrabilidad de copyright

La jurisprudencia reciente (USPTO, Copyright Office EE.UU.) distingue entre obras generadas autónomamente por IA (no registrables) y obras con intervención humana sustancial (registrables). Este proyecto califica como lo segundo: el autor humano dirigió la arquitectura, seleccionó los componentes, revisó cada línea y tomó todas las decisiones de diseño.

5.4 Marco jurisdiccional aplicable

Jurisdicción	Marco legal	Status
Chile	Ley 17.336 (Propiedad Intelectual)	No prohíbe herramientas generativas. Autoría atribuida al creador humano.
Unión Europea	Directiva 2009/24/CE + AI Act (2024)	No atribuye autoría a la herramienta. AI Act regula riesgo del sistema, no propiedad del output.
Estados Unidos	Copyright Act + caso Thaler v. Perlmutter	Autoría requiere intervención humana sustancial, presente en este caso.
Estonia	Autoriõiguse seadus (Ley de Copyright)	Alineada con directiva EU.

5.5 Protección como secreto comercial

Independientemente del análisis de copyright, el codebase está protegido como secreto comercial mediante: repositorio privado, acceso restringido, y obligación contractual de confidencialidad (NDA) para cualquier tercero con acceso.

6. Verificabilidad independiente

Pruebas formales (Lean 4)

Las demostraciones matemáticas en `formal/` son verificables mecánicamente por el kernel de Lean 4. La validez de una prueba formal no depende de quién la escribió sino de que el verificador la acepte. Ejecutar `lean --run` sobre los archivos reproduce la verificación de forma independiente y determinista.

Tests automatizados

El repositorio contiene suites de pruebas unitarias, de integración y E2E ejecutables con `cargo test`. Los resultados son reproducibles independientemente del origen del código.

7. Metodología de desarrollo: IA como amplificador

7.1 Modelo de trabajo

Claude Code fue utilizado como herramienta de pair-programming bajo un modelo de dirección humana continua. La dinámica de trabajo fue:

1. **El autor humano define qué construir** — requisito, contexto de dominio, estándar a cumplir.
2. **Claude genera una propuesta de implementación** — código, tests, documentación.
3. **El autor humano revisa, rechaza o acepta** — cada cambio fue evaluado antes de commit.
4. **Iteración** — correcciones, refinamientos y redirección hasta cumplir el estándar requerido.

Claude no operó de forma autónoma en ningún momento. No tuvo acceso independiente al repositorio, no ejecutó commits sin revisión, y no tomó decisiones arquitectónicas.

7.2 Contribución humana: qué aportó el autor que la herramienta no puede aportar

Conocimiento de dominio interdisciplinario

El proyecto integra criptografía post-cuántica (NIST FIPS 203/204/205), protocolos de consenso BFT (HotStuff), firma electrónica legal (Chile Ley 19.799, EU eIDAS, US ESIGN), estándares de certificación (FIPS 140-3, EA-103, ETSI EN 319 401/411/412/421/422), y verificación formal (Lean 4). Esta combinación de dominios requiere conocimiento especializado para formular las preguntas correctas y evaluar las respuestas.

Un modelo de lenguaje puede generar código para ML-DSA-65 si se le pide. No puede decidir que ML-DSA-65 es el algoritmo correcto para firma avanzada post-cuántica en el contexto de eIDAS, ni que debe combinarse con ECDSA en modo híbrido para compatibilidad transicional, ni que los vectores de validación deben seguir el formato ACVP del NIST.

Criterio de rechazo

El autor rechazó activamente propuestas de Claude cuando:

- Introducían abstracciones innecesarias (interfaces con una sola implementación, factories para un solo producto).
- No cumplían estándares de compliance (ej. uso de SHA-256 donde FIPS requiere SHA-3).
- Violaban convenciones del proyecto (ej. firmas como [u8; 64] en lugar de Vec<u8> para soportar múltiples algoritmos).
- Proponían dependencias externas para lo que stdlib o una dependencia existente ya resolvía.
- Generaban código sobrediseñado para requisitos que no existían.

Cada rechazo fue una decisión arquitectónica. La acumulación de estas decisiones define la identidad técnica del sistema.

Coherencia arquitectónica a lo largo del tiempo

Claude no tiene memoria entre sesiones. Cada sesión comienza sin contexto previo. La coherencia del sistema — 36 módulos, 289 archivos Rust, convenciones consistentes — fue mantenida por el autor humano:

Decisión arquitectónica	Impacto sistémico
Firmas como <code>Vec<u8></code>	Soporta Ed25519 (64B), ML-DSA-65 (3309B) y SLH-DSA (7856B) sin refactor
<code>SigningAlgorithm</code> en toda estructura firmada	Verificación agnóstica al algoritmo en todos los módulos
DID canónico <code>did:goya:{hex[..16]}</code>	Identidad consistente entre API, storage, P2P y compliance
Crypto boundary en crate separado	FIPS 140-3 requiere módulo criptográfico aislado
<code>BlockStore</code> trait para storage	Swap entre <code>MemoryStore</code> (tests) y <code>RocksDB</code> (producción) sin cambios en lógica

Estas decisiones fueron tomadas una vez y sostenidas a lo largo de 791 commits. Sin intervención humana activa, la coherencia se degrada: Claude en una sesión posterior podría proponer una convención diferente si no se le indica la existente.

Dirección de verificación formal

Las pruebas en Lean 4 (`formal/`) no fueron generadas por un prompt genérico. El autor definió:

- Qué propiedades probar (no-fork BFT, correctness de cuarentena, FSM FIPS).
- Qué axiomas aceptar y cuáles demostrar.
- Cómo estructurar los teoremas para que sean verificables por el kernel.

El kernel de Lean 4 verificó las pruebas. Pero la selección de qué probar y por qué es conocimiento humano de dominio.

7.3 Replicabilidad: por qué otro developer + Claude no produce este sistema

Factor	Descripción
Dominio	PQC + BFT + derecho digital + certificación FIPS/ETSI es un cruce de especialidades poco común. Sin este conocimiento, los prompts son genéricos y el output es un prototipo básico.

Factor	Descripción
Decisiones acumuladas	791 commits de decisiones arquitectónicas coherentes. Sin este historial internalizado, cada sesión nueva de Claude puede divergir de las convenciones establecidas.
Criterio de calidad	Saber qué rechazar requiere experiencia. Un developer junior acepta todo lo que Claude propone y termina con un sistema incoherente.
Estándares de compliance	EA-103, FIPS 140-3, ETSI EN 319 4xx no son requisitos que se derivan de un prompt. Requieren lectura de los estándares originales y mapeo manual al sistema.
Verificación formal	Seleccionar propiedades a demostrar requiere criterio matemático y conocimiento de qué garantías importan para certificación.

7.4 Analogía

El modelo de lenguaje es comparable a herramientas profesionales en otras disciplinas:

Disciplina	Herramienta	Lo que produce	Lo que no reemplaza
Arquitectura	AutoCAD / Revit	Planos técnicos	Criterio estructural del arquitecto
Derecho	Westlaw / plantillas legales	Borradores de contratos	Estrategia legal del abogado
Ingeniería de software	Claude Code	Código funcional	Arquitectura, dominio, criterio de calidad

En los tres casos, la herramienta amplifica la capacidad del profesional. No reemplaza el conocimiento que determina qué construir, por qué, y con qué estándar.

8. Auditoría de dependencias

Todas las dependencias provienen de crates.io (registro público de Rust). Licencias presentes en el árbol de dependencias:

Licencia	Tipo	Riesgo comercial
MIT	Permisiva	Ninguno
Apache-2.0	Permisiva	Ninguno

Licencia	Tipo	Riesgo comercial
BSD-2-Clause, BSD-3-Clause	Permisiva	Ninguno
ISC, Zlib, Unlicense, 0BSD, MIT-0, CC0-1.0	Permisiva	Ninguno
MPL-2.0	Copyleft débil (solo archivos modificados del crate original)	Ninguno si no se modifica el crate
CDLA-Permissive-2.0	Permisiva (datos)	Ninguno
Unicode-3.0	Permisiva	Ninguno
BSL-1.0 (Boost)	Permisiva	Ninguno

No se encontró ninguna dependencia bajo GPL, AGPL, SSPL ni ninguna licencia copyleft fuerte. No existe riesgo de contaminación de licencia sobre el código propietario.

No se incorporó código fuente de repositorios de terceros fuera del sistema de dependencias de Cargo. Todo el código en `src/` y `crates/` es original del proyecto.

9. Resumen ejecutivo para inversores

Todo el código de Goya Ledger, LexChain y las pruebas formales en Lean 4 fue desarrollado con Claude (Anthropic) como herramienta de pair-programming bajo dirección humana única. La herramienta amplificó la capacidad del autor; no reemplazó el conocimiento de dominio, las decisiones arquitectónicas ni el criterio de calidad que definen el sistema. No existe riesgo de reclamo de propiedad intelectual por parte del proveedor de IA ni de terceros. La propiedad del output pertenece al autor conforme a los ToS de Anthropic. Todas las dependencias están bajo licencias permisivas (MIT, Apache-2.0, BSD). No se incorporó código de repositorios ajenos. Las pruebas formales son verificables mecánicamente. El codebase está protegido como secreto comercial.

Documento preparado para due diligence. Actualizar ante cambios en herramientas utilizadas o en los Términos de Servicio del proveedor.